

Week 1 Assignment — Solutions & Code Documentation

Topic: Arrays, Strings Traversal, Nested Loops & Best Practices | **Branch:** WEEK1

Submission Guidelines & Best Programming Practices Followed:

- **Variables:** Descriptive naming distinguishing constants, inputs, and results.
- **Modular Methods:** All core logic implemented in standalone methods; main() handles I/O only.
- **Collections Restriction Respected:** Problem 1 uses arrays and nested loops only without Collections classes.
- **Naming Conventions:** Proper camelCase for methods/variables, PascalCase for classes.
- **Exception Handling:** Try-catch-finally blocks with proper resource cleanup for scanners.
- **String & Array Processing:** Efficient string traversal with charAt() and StringBuilder.

Problem 1: The Exam Hall Seat Duplication Checker

Scenario: Scans assigned seat numbers in an exam hall and flags duplicates using arrays and loops only (no Collections).
Signature: void checkDuplicateSeats(int[] seatNumbers)
Concepts: Arrays, nested loops, duplicate detection, conditional logic, basic output formatting.

Sample Input	Expected / Actual Execution Output
{101, 102, 103, 102, 105}	Duplicate Seat Number Found: 102
{101, 102, 103, 104, 105}	No Duplicate Seats Found

Source Code (PROGRAM1.java):

```
import java.util.Scanner;

/**
 * Week 1 - Problem 1: The Exam Hall Seat Duplication Checker
 * Checks for duplicate seat numbers assigned in an exam hall using arrays and loops only (no Collections).
 */
public class PROGRAM1 {

    /**
     * Scans the array of seat numbers and prints duplicates or confirms uniqueness.
     *
     * @param seatNumbers array of integer seat numbers
     */
    public static void checkDuplicateSeats(int[] seatNumbers) {
        if (seatNumbers == null || seatNumbers.length == 0) {
            System.out.println("No Duplicate Seats Found");
            return;
        }

        boolean foundAnyDuplicate = false;
        int totalSeats = seatNumbers.length;

        for (int i = 0; i < totalSeats; i++) {
            // Check if seatNumbers[i] has already been seen earlier to avoid printing the same duplicate multiple times
            boolean alreadyReported = false;
            for (int k = 0; k < i; k++) {
                if (seatNumbers[k] == seatNumbers[i]) {
                    alreadyReported = true;
                    break;
                }
            }
            if (alreadyReported) {
                continue;
            }

            // Check if seatNumbers[i] appears again in the remaining elements
```

```
        for (int j = i + 1; j < totalSeats; j++) {
            if (seatNumbers[i] == seatNumbers[j]) {
                System.out.println("Duplicate Seat Number Found: " + seatNumbers[i]);
                foundAnyDuplicate = true;
                break;
            }
        }
    }

    if (!foundAnyDuplicate) {
        System.out.println("No Duplicate Seats Found");
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    try {
        System.out.print("Enter number of assigned seats: ");
        if (scanner.hasNextInt()) {
            int seatCount = scanner.nextInt();
            int[] assignedSeats = new int[seatCount];
            System.out.println("Enter " + seatCount + " seat numbers:");
            for (int i = 0; i < seatCount; i++) {
                assignedSeats[i] = scanner.nextInt();
            }
            checkDuplicateSeats(assignedSeats);
        }
    } catch (Exception e) {
        System.err.println("An error occurred while reading seat inputs: " + e.getMessage());
    } finally {
        scanner.close();
    }
}
```

Problem 2: The Typing Speed Test Accuracy Checker

Scenario: Compares typed attempt character-by-character against the original passage, computes accuracy %, and reports first mismatch.

Signature: void checkTypingAccuracy(String original, String typed)

Concepts: String traversal, charAt(), loops, percentage calculation, StringBuilder formatting.

Sample Input	Expected / Actual Execution Output
original="hello world", typed="hello worlt"	Matched: 10/11 Accuracy: 90.91% First Mismatch at position 11 ('d' vs 't')
original="coding", typed="coding"	Matched: 6/6 Accuracy: 100.00% No Mismatches

Source Code (PROGRAM2.java):

```
import java.util.Scanner;

/**
 * Week 1 - Problem 2: The Typing Speed Test Accuracy Checker
 * Compares typed text against original passage character by character and calculates accuracy percentage.
 */
public class PROGRAM2 {

    /**
     * Compares original and typed strings and prints accuracy and first mismatch info.
     *
     * @param original original passage text
     * @param typed     user's typed text
     */
    public static void checkTypingAccuracy(String original, String typed) {
        if (original == null || typed == null) {
            System.out.println("Invalid input: Strings cannot be null.");
            return;
        }

        int originalLength = original.length();
        int typedLength = typed.length();

        // Compare up to the length of the strings
        int totalLength = Math.min(originalLength, typedLength);
        if (totalLength == 0) {
            System.out.println("Matched: 0/0 | Accuracy: 0.00% | No Mismatches");
            return;
        }

        int matchedCount = 0;
        int firstMismatchPosition = -1;
        char originalCharMismatch = ' ';
        char typedCharMismatch = ' ';

        for (int i = 0; i < totalLength; i++) {
            char origChar = original.charAt(i);
            char typedChar = typed.charAt(i);

            if (origChar == typedChar) {
                matchedCount++;
            } else if (firstMismatchPosition == -1) {
                firstMismatchPosition = i + 1; // 1-based index position
                originalCharMismatch = origChar;
                typedCharMismatch = typedChar;
            }
        }

        double accuracyPercentage = ((double) matchedCount / totalLength) * 100.0;

        StringBuilder reportBuilder = new StringBuilder();
    }
}
```

```
        reportBuilder.append("Matched: ").append(matchedCount).append("/").append(totalLength)
            .append(" | Accuracy: ").append(String.format("%.2f", accuracyPercentage)).append("%");

        if (firstMismatchPosition != -1) {
            reportBuilder.append(" | First Mismatch at position ").append(firstMismatchPosition)
                .append(" ('").append(originalCharMismatch).append("' vs '")
                .append(typedCharMismatch).append("')");
        } else if (originalLength != typedLength) {
            reportBuilder.append(" | First Mismatch at position ").append(totalLength + 1)
                .append(" (Length difference)");
        } else {
            reportBuilder.append(" | No Mismatches");
        }

        System.out.println(reportBuilder.toString());
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Enter original passage: ");
            String originalText = scanner.nextLine();
            System.out.print("Enter typed text: ");
            String typedText = scanner.nextLine();
            checkTypingAccuracy(originalText, typedText);
        } catch (Exception e) {
            System.err.println("An error occurred while evaluating typing accuracy: " + e.getMessage());
        } finally {
            scanner.close();
        }
    }
}
```

Problem 3: The Traffic Signal Streak Analyzer

Scenario: Scans daily signal sequence log ('R','Y','G') and identifies the longest continuous streak of the same color.

Signature: void findLongestStreak(String signalLog)

Concepts: String traversal, character comparison, tracking running streak and maximum.

Sample Input	Expected / Actual Execution Output
"RRGGGYRR"	Longest Streak: 'G' repeated 3 times
"RRRRYYGG"	Longest Streak: 'R' repeated 4 times

Source Code (PROGRAM3.java):

```
import java.util.Scanner;

/**
 * Week 1 - Problem 3: The Traffic Signal Streak Analyzer
 * Scans a sequence of signal readings and finds the longest continuous streak of the same color.
 */
public class PROGRAM3 {

    /**
     * Finds and prints the color and length of the longest continuous streak.
     *
     * @param signalLog sequence string containing signal readings (e.g., 'R', 'Y', 'G')
     */
    public static void findLongestStreak(String signalLog) {
        if (signalLog == null || signalLog.isEmpty()) {
            System.out.println("Invalid or empty signal log.");
            return;
        }

        char longestColor = signalLog.charAt(0);
        int maxStreakLength = 1;

        char currentColor = signalLog.charAt(0);
        int currentStreakLength = 1;

        for (int i = 1; i < signalLog.length(); i++) {
            char reading = signalLog.charAt(i);
            if (reading == currentColor) {
                currentStreakLength++;
            } else {
                if (currentStreakLength > maxStreakLength) {
                    maxStreakLength = currentStreakLength;
                    longestColor = currentColor;
                }
                currentColor = reading;
                currentStreakLength = 1;
            }
        }

        // Check the final streak
        if (currentStreakLength > maxStreakLength) {
            maxStreakLength = currentStreakLength;
            longestColor = currentColor;
        }

        System.out.println("Longest Streak: '" + longestColor + "' repeated " + maxStreakLength + " times");
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Enter signal log: ");
            if (scanner.hasNextLine()) {

```

```
        String logInput = scanner.nextLine().trim();
        findLongestStreak(logInput);
    }
} catch (Exception e) {
    System.err.println("An error occurred: " + e.getMessage());
} finally {
    scanner.close();
}
}
```

Problem 4: The Warehouse Inventory Balancer

Scenario: Computes item totals across storage sections A & B, determines balance, and identifies highest-quantity item index.

Signature: void analyzeInventory(int[] sectionA, int[] sectionB)

Concepts: Arrays, loops, sum accumulation, conditional comparison, tracking maximum with section and index.

Sample Input	Expected / Actual Execution Output
sectionA={20,15,30}, sectionB={25,10,30}	Section A Total: 65 Section B Total: 65 Status: Balanced Highest Quantity: 30 (Section A, Item 3)

Source Code (PROGRAM4.java):

```
import java.util.Scanner;

/**
 * Week 1 - Problem 4: The Warehouse Inventory Balancer
 * Computes section totals, evaluates inventory balance, and tracks the maximum quantity item.
 */
public class PROGRAM4 {

    /**
     * Computes totals for section A and section B, compares balances,
     * and locates the overall highest quantity item.
     *
     * @param sectionA array of item quantities in Section A
     * @param sectionB array of item quantities in Section B
     */
    public static void analyzeInventory(int[] sectionA, int[] sectionB) {
        if (sectionA == null || sectionB == null || sectionA.length == 0 || sectionB.length == 0) {
            System.out.println("Invalid inventory data.");
            return;
        }

        int lengthA = sectionA.length;
        int lengthB = sectionB.length;

        int totalA = 0;
        int totalB = 0;

        int maxQuantity = Integer.MIN_VALUE;
        String maxSection = "";
        int maxIndex = -1; // 1-based item index

        // Scan Section A
        for (int i = 0; i < lengthA; i++) {
            totalA += sectionA[i];
            if (sectionA[i] > maxQuantity) {
                maxQuantity = sectionA[i];
                maxSection = "Section A";
                maxIndex = i + 1; // 1-based index (e.g. Item 1, Item 2...)
            }
        }

        // Scan Section B
        for (int i = 0; i < lengthB; i++) {
            totalB += sectionB[i];
            if (sectionB[i] > maxQuantity) {
                maxQuantity = sectionB[i];
                maxSection = "Section B";
                maxIndex = i + 1;
            }
        }

        String status = (totalA == totalB) ? "Balanced" : "Not Balanced";

        System.out.println("Section A Total: " + totalA + " | Section B Total: " + totalB +
```

```
        " | Status: " + status + " | Highest Quantity: " + maxQuantity +  
        " (" + maxSection + ", Item " + maxIndex + ")");  
    }  
  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        try {  
            System.out.print("Enter number of categories per section: ");  
            if (scanner.hasNextInt()) {  
                int count = scanner.nextInt();  
                int[] sectionA = new int[count];  
                int[] sectionB = new int[count];  
  
                System.out.println("Enter " + count + " quantities for Section A:");  
                for (int i = 0; i < count; i++) {  
                    sectionA[i] = scanner.nextInt();  
                }  
  
                System.out.println("Enter " + count + " quantities for Section B:");  
                for (int i = 0; i < count; i++) {  
                    sectionB[i] = scanner.nextInt();  
                }  
  
                analyzeInventory(sectionA, sectionB);  
            }  
        } catch (Exception e) {  
            System.err.println("An error occurred while analyzing inventory: " + e.getMessage());  
        } finally {  
            scanner.close();  
        }  
    }  
}
```

Problem 5: The Movie Review Word Length Profiler

Scenario: Scans review text, splits into individual words, and categorizes lengths: Short (1-4), Medium (5-8), Long (9+).

Signature: void classifyWordLengths(String review)

Concepts: String splitting (split()), loops, conditional logic, counting and categorization.

Sample Input	Expected / Actual Execution Output
"This movie was absolutely fantastic and thrilling"	Short: 3 Medium: 1 Long: 3

Source Code (PROGRAM5.java):

```
import java.util.Scanner;

/**
 * Week 1 - Problem 5: The Movie Review Word Length Profiler
 * Scans a review text, splits into individual words, and categorizes lengths:
 * Short (1-4 letters), Medium (5-8 letters), Long (9+ letters).
 */
public class PROGRAM5 {

    /**
     * Splits review text into words, categorizes each word by character length, and prints counts.
     *
     * @param review movie review string
     */
    public static void classifyWordLengths(String review) {
        if (review == null || review.trim().isEmpty()) {
            System.out.println("Short: 0 | Medium: 0 | Long: 0");
            return;
        }

        // Split review into individual words using whitespace regex
        String[] words = review.trim().split("\\s+");

        int shortCount = 0;
        int mediumCount = 0;
        int longCount = 0;

        for (String rawWord : words) {
            // Count letter characters in the word (strip punctuation if any)
            int letterLength = 0;
            for (int i = 0; i < rawWord.length(); i++) {
                if (Character.isLetter(rawWord.charAt(i))) {
                    letterLength++;
                }
            }

            // If no letters found, fallback to total length
            int effectiveLength = (letterLength > 0) ? letterLength : rawWord.length();

            if (effectiveLength >= 1 && effectiveLength <= 4) {
                shortCount++;
            } else if (effectiveLength >= 5 && effectiveLength <= 8) {
                mediumCount++;
            } else if (effectiveLength >= 9) {
                longCount++;
            }
        }

        System.out.println("Short: " + shortCount + " | Medium: " + mediumCount + " | Long: " + longCount);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
```

```
        System.out.print("Enter movie review: ");
        if (scanner.hasNextLine()) {
            String reviewText = scanner.nextLine();
            classifyWordLengths(reviewText);
        }
    } catch (Exception e) {
        System.err.println("An error occurred while profiling review: " + e.getMessage());
    } finally {
        scanner.close();
    }
}
}
```